JEU PONG AVEC PYXEL

OBJECTIF : Développer un jeu Pong à 2 joueurs où deux raquettes se déplacent verticalement pour renvoyer une balle. Si la balle passe derrière une raquette, l'autre joueur marque un point !

ÉTAPE 1 : Initialiser le terrain et les raquettes

Le terrain mesure 160 x 120 pixels.

- Raquettes : 4 x 20 pixels.
- Raquette J1 (Gauche) : x = 5, y = 50
- Raquette J2 (Droite) : x = 151, y = 50

À toi de jouer :

1. Crée un fichier "pong.py".
2. Complète le code d'initialisation :

```
import pyxel

SCREEN_WIDTH = 160 SCREEN_HEIGHT = 120 PADDLE_WIDTH = 4 PADDLE_HEIGHT =
20 PADDLE_SPEED = 3

class PongGame: def init(self): pyxel.init(SCREEN_WIDTH, SCREEN_HEIGHT,
title="Pyxel Pong")

    self.p1_y = (SCREEN_HEIGHT - PADDLE_HEIGHT) // 2
    self.p2_y = (SCREEN_HEIGHT - PADDLE_HEIGHT) // 2

    pyxel.run(self.update, self.draw)

def update(self):
    # 1. Déplacement J1 (Z / S)
    if pyxel.btn(pyxel.KEY_Z) and self.p1_y > 0:
        self.p1_y -= PADDLE_SPEED
    if pyxel.btn(pyxel.KEY_S) and self.p1_y < SCREEN_HEIGHT - PADDLE_HEIGHT:
        self.p1_y += PADDLE_SPEED

    # 2. Déplacement J2 (Haut / Bas)
    if pyxel.btn(pyxel.KEY_UP) and self.p2_y > 0:
        self.p2_y -= PADDLE_SPEED
    if pyxel.btn(pyxel.KEY_DOWN) and self.p2_y < SCREEN_HEIGHT - PADDLE_HEIGHT:
        self.p2_y += PADDLE_SPEED

def draw(self):
    pyxel.cls(0)

    # Filet
    for y in range(0, SCREEN_HEIGHT, 8):
        pyxel.line(SCREEN_WIDTH // 2, y, SCREEN_WIDTH // 2, y + 4, 13)

    # Raquettes
    pyxel.rect(5, self.p1_y, PADDLE_WIDTH, PADDLE_HEIGHT, 11)
    pyxel.rect(151, self.p2_y, PADDLE_WIDTH, PADDLE_HEIGHT, 12)

PongGame()
```

ÉTAPE 2 : Ajouter la balle et ses déplacements

La balle est un carré de 4x4 pixels avec une position (x, y) et une vitesse (dx, dy).

À toi de jouer :

1. Dans **init**, initialise la balle au centre :

```
self.ball_x = SCREEN_WIDTH // 2
self.ball_y = SCREEN_HEIGHT // 2
self.ball_dx = 2
self.ball_dy = 1.5
```

2. Dans update(), fais avancer la balle : A faire !
3. Gère les rebonds sur les murs du haut et du bas :

   Si self.ball_y <= 0 ou self.ball_y >= SCREEN_HEIGHT - 4 -> inverse self.ball_dy.

4. Dans draw(), affiche la balle :

```
pyxel.rect(self.ball_x, self.ball_y, 4, 4, 7)
```

ÉTAPE 3 : Collisions avec les raquettes

La balle doit rebondir si elle touche une raquette.

À toi de jouer : Ajoute ces conditions de collision dans update() :

Collision Raquette Gauche (J1)

```
if self.ball_dx < 0 and self.ball_x <= 5 + PADDLE_WIDTH and self.p1_y <=
self.ball_y + 4 and self.ball_y <= self.p1_y + PADDLE_HEIGHT: self.ball_dx = -
self.ball_dx
```

Collision Raquette Droite (J2)

A faire !

ÉTAPE 4 : Gestion des points et réinitialisation

Si la balle sort de l'écran à gauche ou à droite, un point est marqué.

À toi de jouer :

1. Ajoute self.score_p1 = 0 et self.score_p2 = 0 dans le constructeur.
2. Crée une méthode reset_ball(self) qui replace la balle au centre.
3. Dans update(), vérifie si self.ball_x < 0 (point pour J2) ou self.ball_x > SCREEN_WIDTH (point pour J1).
4. Affiche les scores dans draw() avec pyxel.text().